ARM-CBA: Tuning & Feature Selection Experiments

CBA (Liu et al., 1998) faces a challenge that LR and RF don't: its rule mining step relies on a minimum support threshold, meaning rare classes can be mathematically invisible if they don't appear frequently enough in training data. At natural class frequencies, Signal (2.5%) can only produce rules with at most 2.5% support — below most useful thresholds.

This notebook solves that through three experiments:

1. **Remove Accident Type** — confirm whether CBA can generate meaningful rules without the dominant shortcut feature
2. **Undersample inside CV folds** — balance class distribution before rule mining so minority causes get equal access to support thresholds
3. **Tune thresholds on undersampled data** — find the support/confidence configuration that best balances rule coverage against reliability

Feature decisions (Track Type over Signalization, RUCC_Metro_Adjacency over RUCC_2023) were established in LR tuning and apply unchanged here.

Reference

Liu, B., Hsu, W., & Ma, Y. (1998). Integrating Classification and Association Rule Mining. *Proceedings of the Fourth International Conference on Knowledge Discovery and Data Mining (KDD-98)*, 80-86.

Setup

In [21]:
```
import pandas as pd
import numpy as np
import json
import joblib
import warnings

from mlxtend.frequent_patterns import apriori, association_rules
from sklearn.base import BaseEstimator, ClassifierMixin
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import (
    classification_report, f1_score, confusion_matrix,
    precision_score, recall_score
)
```

```
import matplotlib.pyplot as plt
import matplotlib.style as style
import seaborn as sns

warnings.filterwarnings('ignore', category=FutureWarning)
warnings.filterwarnings('ignore', category=DeprecationWarning)
style.use('tableau-colorblind10')

plt.rcParams['figure.dpi'] = 120
plt.rcParams['font.size'] = 10

RANDOM_SEED = 521
np.random.seed(RANDOM_SEED)

CAUSE_ORDER = ['E', 'H', 'M', 'S', 'T']
CAUSE_LABELS = {
    'E': 'Environmental', 'H': 'Human Factor',
    'M': 'Mechanical', 'S': 'Signal', 'T': 'Track'
}
```

CBA Classifier

Same implementation from the baseline notebook. Wraps ARM rule mining and first-match classification in a sklearn-compatible class with `fit()` and `predict()` methods. See baseline notebook for detailed step-by-step comments.

One addition: an optional `undersample` parameter. When enabled, the fit step balances the training data before mining rules by downsampling majority classes to match the smallest class count.

In [22]:
```
class CBAClassifier(BaseEstimator, ClassifierMixin):
    """
    Classification Based on Associations (Liu et al., 1998).

    Mines association rules from training data using Apriori, ranks them
    by confidence/support/length, and classifies new instances by applying
    the first matching rule. Falls back to majority class when no rule matc

    Parameters
    ----------
    min_support : float
        Minimum support threshold for Apriori.
    min_confidence : float
        Minimum confidence for filtering rules.
    min_lift : float
        Minimum lift for filtering rules.
    max_rule_length : int
        Maximum items in a frequent itemset.
    undersample : bool
        If True, balance training data before mining rules by downsampling
        majority classes to match the smallest class count.
    """
```

```python
    def __init__(self, min_support=0.02, min_confidence=0.70,
                 min_lift=1.5, max_rule_length=4, undersample=False):
        self.min_support = min_support
        self.min_confidence = min_confidence
        self.min_lift = min_lift
        self.max_rule_length = max_rule_length
        self.undersample = undersample

    def fit(self, X, y):
        """
        Mine association rules from training data and build ranked rule list
        If undersample=True, balance classes before mining.
        """
        X_fit = X.copy()
        y_fit = y.copy()

        # Optional undersampling: balance classes before rule mining
        # This gives minority classes (Signal, Environmental) a fair chance
        # of meeting support thresholds by making them proportionally larger
        if self.undersample:
            min_count = y_fit.value_counts().min()
            balanced_idx = []
            for cls in y_fit.unique():
                cls_idx = y_fit[y_fit == cls].index.tolist()
                sampled = np.random.choice(cls_idx, size=min_count, replace=
                balanced_idx.extend(sampled)

            np.random.shuffle(balanced_idx)
            X_fit = X_fit.loc[balanced_idx].reset_index(drop=True)
            y_fit = y_fit.loc[balanced_idx].reset_index(drop=True)

        # One-hot encode features into binary transaction format
        X_ohe = pd.get_dummies(X_fit, columns=X_fit.columns.tolist(), dtype=

        # One-hot encode target and append
        y_ohe = pd.get_dummies(y_fit, prefix='Cause', dtype=bool)
        self.cause_columns_ = y_ohe.columns.tolist()
        df_arm = pd.concat([X_ohe, y_ohe], axis=1)

        # Run Apriori
        frequent_itemsets = apriori(
            df_arm,
            min_support=self.min_support,
            use_colnames=True,
            max_len=self.max_rule_length
        )

        if len(frequent_itemsets) == 0:
            self.rules_ = pd.DataFrame()
            self.default_class_ = y_fit.mode()[0]
            self.n_rules_ = 0
            return self

        # Generate and filter rules
        rules = association_rules(frequent_itemsets, metric='lift',
                                  min_threshold=self.min_lift)
```

```
        rules = rules[rules['confidence'] >= self.min_confidence]

        # Filter for Class Association Rules
        car_mask = rules['consequents'].apply(
            lambda x: len(x) == 1 and any(item in self.cause_columns_ for it
        )
        cars = rules[car_mask].copy()

        # Rank: confidence desc, support desc, length asc
        cars['antecedent_length'] = cars['antecedents'].apply(len)
        cars = cars.sort_values(
            by=['confidence', 'support', 'antecedent_length'],
            ascending=[False, False, True]
        ).reset_index(drop=True)

        cars['predicted_cause'] = cars['consequents'].apply(
            lambda x: list(x)[0].replace('Cause_', '')
        )

        self.rules_ = cars
        self.n_rules_ = len(cars)
        self.feature_columns_ = X_ohe.columns.tolist()

        # Default rule: majority class from training data (Liu et al., 1998)
        # Uses original y (not undersampled) to reflect true class distribut
        self.default_class_ = y.mode()[0]

        return self

    def predict(self, X):
        """
        Classify each row by first matching rule. Default to majority class.
        """
        X_ohe = pd.get_dummies(X, columns=X.columns.tolist(), dtype=bool)
        X_ohe = X_ohe.reindex(columns=self.feature_columns_, fill_value=Fals

        predictions = []

        if self.n_rules_ == 0:
            return np.array([self.default_class_] * len(X))

        # Pre-extract for speed
        rule_antecedents = []
        rule_predictions = []
        for _, rule in self.rules_.iterrows():
            antes = [a for a in rule['antecedents'] if a in self.feature_col
            rule_antecedents.append(antes)
            rule_predictions.append(rule['predicted_cause'])

        for idx in range(len(X_ohe)):
            row = X_ohe.iloc[idx]
            matched = False

            for rule_idx in range(self.n_rules_):
                antes = rule_antecedents[rule_idx]
                if not antes:
```

```
                    continue
                if all(row[a] for a in antes):
                    predictions.append(rule_predictions[rule_idx])
                    matched = True
                    break

            if not matched:
                predictions.append(self.default_class_)

        return np.array(predictions)

print("CBAClassifier defined (with undersample option).")
```

```
CBAClassifier defined (with undersample option).
```

Helper Functions

In [23]:

```
def run_cba_experiment(cba, X, y, n_splits=5):
    """
    Run stratified k-fold CV for CBA and return scores and metrics.
    """
    skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=RAND

    scores = []
    rule_counts = []
    last_y_val = None
    last_y_pred = None

    for fold, (train_idx, val_idx) in enumerate(skf.split(X, y), 1):
        X_tr = X.iloc[train_idx]
        X_val = X.iloc[val_idx]
        y_tr = y.iloc[train_idx]
        y_val = y.iloc[val_idx]

        cba.fit(X_tr, y_tr)
        y_pred = cba.predict(X_val)
        f1 = f1_score(y_val, y_pred, average='weighted')
        scores.append(f1)
        rule_counts.append(cba.n_rules_)

        last_y_val = y_val
        last_y_pred = y_pred

    precision_per_class = precision_score(last_y_val, last_y_pred, average=N
                                          labels=CAUSE_ORDER, zero_division=
    recall_per_class = recall_score(last_y_val, last_y_pred, average=None,
                                    labels=CAUSE_ORDER, zero_division=0)
    f1_per_class = f1_score(last_y_val, last_y_pred, average=None,
                            labels=CAUSE_ORDER, zero_division=0)

    per_class = pd.DataFrame({
        'Cause': CAUSE_ORDER,
        'Precision': precision_per_class.round(3),
        'Recall': recall_per_class.round(3),
        'F1': f1_per_class.round(3)
```

```
    })

    return {
        'scores': [float(s) for s in scores],
        'mean': float(np.mean(scores)),
        'std': float(np.std(scores)),
        'rule_counts': rule_counts,
        'per_class_metrics': per_class,
        'last_y_val': last_y_val,
        'last_y_pred': last_y_pred
    }


def save_experiment(results, features, experiment_name, filepath, **extra_fi
    """
    Save experiment results to JSON.
    """
    output = {
        'experiment': experiment_name,
        'model': 'CBA (Classification Based on Associations)',
        'features': {
            'categorical': features['categorical'],
            'numeric': features['numeric'],
            'total': len(features['categorical']) + len(features['numeric'])
        },
        'cv_folds': 5,
        'cv_scores': results['scores'],
        'cv_mean': results['mean'],
        'cv_std': results['std'],
        'rule_counts': results['rule_counts'],
        'per_class_metrics': results['per_class_metrics'].to_dict('records')
        'random_seed': RANDOM_SEED
    }
    output.update(extra_fields)

    with open(filepath, 'w') as f:
        json.dump(output, f, indent=4)

    print(f"Saved: {filepath}")


def compare_experiments(names, filepaths):
    """
    Load multiple experiment JSONs and display side-by-side comparison.
    """
    print("=" * 70)
    print(f"{'Experiment':<40} {'CV Mean':>10} {'CV Std':>10} {'Features':>8
    print("-" * 70)

    for name, fp in zip(names, filepaths):
        with open(fp, 'r') as f:
            data = json.load(f)

        mean = data.get('cv_mean', 0)
        std = data.get('cv_std', 0)
```

```
        if isinstance(data.get('features'), dict):
            n_feat = data['features']['total']
        elif isinstance(data.get('features'), list):
            n_feat = len(data['features'])
        else:
            n_feat = '?'

        print(f"{name:<40} {mean:>10.4f} {std:>10.4f} {n_feat:>8}")

    print("=" * 70)


def plot_cv_comparison(experiments, title='CV Comparison'):
    """
    Plot per-fold F1 scores for multiple experiments.
    """
    fig, ax = plt.subplots(figsize=(8, 5))
    folds = np.arange(1, 6)

    markers = ['o', 's', '^', 'D', 'v']
    linestyles = ['-', '--', '-.', ':', '-']

    for i, (label, scores) in enumerate(experiments):
        mean = np.mean(scores)
        ax.plot(folds, scores, marker=markers[i % len(markers)],
                linewidth=2, linestyle=linestyles[i % len(linestyles)],
                label=f'{label} (mean={mean:.4f})')
        ax.axhline(y=mean, linestyle=':', alpha=0.3)

    ax.set_xlabel('Fold')
    ax.set_ylabel('Weighted F1-Score')
    ax.set_title(title)
    ax.set_xticks(folds)
    ax.legend()
    plt.tight_layout()
    plt.show()


def plot_per_class_comparison(results_list, labels, title='Per-Class F1 Comp
    """
    Side-by-side per-class F1 bars for multiple experiments.
    """
    fig, ax = plt.subplots(figsize=(8, 5))

    x = np.arange(len(CAUSE_ORDER))
    n = len(results_list)
    width = 0.8 / n
    hatches = ['//', '..', 'xx', '\\\\', '++']

    for i, (df, label) in enumerate(zip(results_list, labels)):
        offset = (i - n/2 + 0.5) * width
        bars = ax.bar(x + offset, df['F1'], width, label=label)
        for bar in bars:
            bar.set_hatch(hatches[i % len(hatches)])

    ax.set_xlabel('Cause Category')
```

```
    ax.set_ylabel('F1-Score')
    ax.set_title(title)
    ax.set_xticks(x)
    ax.set_xticklabels([f"{c}" for c in CAUSE_ORDER])
    ax.legend()
    ax.set_ylim(0, 1.0)
    plt.tight_layout()
    plt.show()

print("Helper functions defined.")
```

```
Helper functions defined.
```

Load Training Data

In [24]:
```
df_train = pd.read_csv('../data/splits/train_80.csv', low_memory=False)

target = 'Cause_Category'
y_train = df_train[target].copy()

print(f"Training set: {len(df_train)} rows")
print(f"\nCause_Category distribution:")
print(y_train.value_counts())
print(f"\nSmallest class: {y_train.value_counts().min()} rows "
      f"({y_train.value_counts().min() / len(y_train):.1%})")
```

```
Training set: 27640 rows

Cause_Category distribution:
Cause_Category
H    10863
M     7081
T     6026
E     2977
S      693
Name: count, dtype: int64

Smallest class: 693 rows (2.5%)
```

In [25]:
```
# Load baseline results for comparison
with open('../results/cba/cba_baseline_results.json', 'r') as f:
    baseline_results = json.load(f)

print(f"Baseline CBA (Low Threshold) CV: {baseline_results.get('cv_mean', 'N
```

```
Baseline CBA (Low Threshold) CV: 0.5000738435598553
```

Section 1: Feature Validation

Experiment 1: Remove Accident Type

The baseline CBA (with Accident Type, low thresholds) scored 0.500 weighted F1 — but all of its rules were Accident Type shortcuts. Phase 2 ARM experiments found zero rules at stricter thresholds once Accident Type was removed, suggesting the remaining features couldn't form strong enough associations.

This experiment tests that at the most permissive thresholds (S=0.005, C=0.40, Lift=1.5): does removing Accident Type collapse CBA to a majority-class predictor, or do the 5 environmental features generate usable rules?

In [26]:
```
# Experiment 1: 5 features, no Accident Type, same low thresholds as baselir
cat_features_exp1 = ['Weather Condition', 'Track Type', 'Visibility', 'Regic
                     'RUCC_Metro_Adjacency']
num_features_exp1 = []

X_exp1 = df_train[cat_features_exp1].copy()

# Use the baseline's best low thresholds
cba_exp1 = CBAClassifier(
    min_support=0.005,
    min_confidence=0.40,
    min_lift=1.5,
    max_rule_length=4,
    undersample=False
)

print("Experiment 1: CBA without Accident Type (no undersampling)")
print("-" * 55)
results_exp1 = run_cba_experiment(cba_exp1, X_exp1, y_train)

for fold, (score, n_rules) in enumerate(zip(results_exp1['scores'], results_
    print(f"  Fold {fold}: Weighted F1 = {score:.4f}  (Rules: {n_rules})")
print(f"\nCV: {results_exp1['mean']:.4f} +/- {results_exp1['std']:.4f}")
```

```
Experiment 1: CBA without Accident Type (no undersampling)
-------------------------------------------------------
  Fold 1: Weighted F1 = 0.4214  (Rules: 65)
  Fold 2: Weighted F1 = 0.4165  (Rules: 62)
  Fold 3: Weighted F1 = 0.4143  (Rules: 64)
  Fold 4: Weighted F1 = 0.4166  (Rules: 61)
  Fold 5: Weighted F1 = 0.4234  (Rules: 60)

CV: 0.4184 +/- 0.0034
```

In [27]:
```
# Examine what rules (if any) were found
if cba_exp1.n_rules_ > 0:
    print(f"Rules mined (last fold): {cba_exp1.n_rules_}")
    print(f"Default class: {cba_exp1.default_class_}")
    print(f"\nRule count by predicted cause:")
    print(cba_exp1.rules_['predicted_cause'].value_counts())
    print(f"\nTop 10 rules:")
    top_rules = cba_exp1.rules_[['antecedents', 'predicted_cause', 'confider
                                 'support', 'lift']].head(10).copy()
    top_rules['antecedents'] = top_rules['antecedents'].apply(lambda x: ', '
    print(top_rules.to_string(index=False))
```

```
else:
    print(f"Rules mined: 0")
    print(f"Default class: {cba_exp1.default_class_}")
    print(f"\nNo rules generated. Model defaults to majority class (H) for a
    print(f"This confirms Phase 2 ARM findings: without Accident Type,")
    print(f"the remaining features cannot meet support thresholds at natural
```

```
Rules mined (last fold): 60
Default class: H

Rule count by predicted cause:
predicted_cause
M    41
T    17
H     2
Name: count, dtype: int64

Top 10 rules:
                                                          antecedents predicte
d_cause  confidence  support     lift
                       Region_West, Track Type_Yard, Visibility_Dark
H    0.606061 0.019899 1.541964
                            Track Type_Main, Weather Condition_Rain
M    0.603806 0.015783 2.356816
             Track Type_Main, Visibility_Dark, Weather Condition_Rain
M    0.600858 0.006331 2.345310
                Region_South, Track Type_Main, Weather Condition_Rain
M    0.595041 0.006512 2.322604
             RUCC_Metro_Adjacency_Metro, Region_West, Track Type_Yard
H    0.594663 0.042330 1.512967
              Track Type_Main, Visibility_Day, Weather Condition_Rain
M    0.590717 0.006331 2.305727
RUCC_Metro_Adjacency_Metro, Track Type_Main, Weather Condition_Rain
M    0.574866 0.009723 2.243856
             RUCC_Metro_Adjacency_Metro, Region_West, Track Type_Main
M    0.555556 0.024421 2.168481
                       Region_South, Track Type_Main, Visibility_Day
M    0.538655 0.038757 2.102513
      Track Type_Industry, Visibility_Day, Weather Condition_Cloudy
T    0.533632 0.005382 2.447558
```

In [28]:

```
save_experiment(
    results_exp1,
    features={'categorical': cat_features_exp1, 'numeric': num_features_exp1
    experiment_name='CBA No AT (no undersampling)',
    filepath='../results/cba/cba_exp_no_accident_type.json',
    thresholds={'min_support': 0.005, 'min_confidence': 0.40, 'min_lift': 1.
    undersample=False,
    notes='60 rules found at permissive thresholds (Phase 2 zero-rule result
)

compare_experiments(
    names=['Baseline (6 feat, w/ AT)', 'Without AT (no undersampling)'],
    filepaths=['../results/cba/cba_baseline_results.json', '../results/cba/c
)
```

```
Saved: ../results/cba/cba_exp_no_accident_type.json
======================================================================
Experiment                                   CV Mean     CV Std Features
----------------------------------------------------------------------
Baseline (6 feat, w/ AT)                      0.5001     0.0032        6
Without AT (no undersampling)                 0.4184     0.0034        5
======================================================================
```

Experiment 1 Findings: ARM Without Accident Type

At permissive thresholds (S=0.005, C=0.40, Lift=1.5), CBA generated **60 rules** without Accident Type — not the zero-rule result seen in Phase 2, which used stricter settings. Overall F1 dropped from 0.500 to 0.418, a smaller fall than LR (0.536→0.409, -0.127) or RF (0.551→0.421, -0.130), partly because CBA's baseline was already lower.

The rules themselves are meaningful: "Rain + Main track → Mechanical" and "Industry track + Cloudy → Track" describe real environmental conditions — not Accident Type shortcuts. That's progress.

The remaining problem: All 60 rules cover only Mechanical (41), Track (17), and Human Factor (2). Zero rules for Signal or Environmental. At natural class frequencies, patterns from a 2.5% class can't accumulate enough support to survive the threshold, no matter how permissive. This motivates Experiment 2.

Experiment 2: Undersampling Inside CV Folds

The idea: Before mining rules in each fold, downsample majority classes to match Signal (~700 rows), creating a balanced 20%/20%/20%/20%/20% distribution. Rules are mined from this balanced data, but predictions and F1 scores are evaluated on the original unbalanced validation fold — so results still reflect real-world performance.

Why it works: ARM's support threshold is a fraction of total rows. When Signal is 2.5% of data, a Signal-predicting rule can have at most 2.5% support — below any useful threshold. After balancing, Signal patterns have the same opportunity to meet support thresholds as Human Factor patterns. Same thresholds, fair access.

What to look for in the results:

- **Rule diversity:** Do we now get rules for all 5 cause categories, not just M and H?
- **Signal and Environmental rules:** These were impossible without undersampling. Do they appear?
- **F1 score:** May go up or down. More rules means better coverage, but rules mined from undersampled data may be less reliable on the natural distribution.
- **Coverage:** What percentage of rows now match a rule vs defaulting?

In [29]:
```
# Experiment 2: Same features, same thresholds, but with undersampling
cba_exp2 = CBAClassifier(
    min_support=0.005,
    min_confidence=0.40,
    min_lift=1.5,
    max_rule_length=4,
    undersample=True
)

print("Experiment 2: CBA without Accident Type (with undersampling)")
print("-" * 55)
results_exp2 = run_cba_experiment(cba_exp2, X_exp1, y_train)

for fold, (score, n_rules) in enumerate(zip(results_exp2['scores'], results_
    print(f"  Fold {fold}: Weighted F1 = {score:.4f}  (Rules: {n_rules})")
print(f"\nCV: {results_exp2['mean']:.4f} +/- {results_exp2['std']:.4f}")
```

```
Experiment 2: CBA without Accident Type (with undersampling)
-------------------------------------------------------
  Fold 1: Weighted F1 = 0.3938  (Rules: 61)
  Fold 2: Weighted F1 = 0.3958  (Rules: 55)
  Fold 3: Weighted F1 = 0.4223  (Rules: 55)
  Fold 4: Weighted F1 = 0.3901  (Rules: 64)
  Fold 5: Weighted F1 = 0.4308  (Rules: 62)

CV: 0.4066 +/- 0.0167
```

In [30]:
```
# Examine rules from undersampled model
if cba_exp2.n_rules_ > 0:
    print(f"Rules mined (last fold): {cba_exp2.n_rules_}")
    print(f"Default class: {cba_exp2.default_class_}")
    print(f"\nRule count by predicted cause:")
    print(cba_exp2.rules_['predicted_cause'].value_counts())
    print(f"\nTop 15 rules:")
    top_rules = cba_exp2.rules_[['antecedents', 'predicted_cause', 'confider
                                 'support', 'lift']].head(15).copy()
    top_rules['antecedents'] = top_rules['antecedents'].apply(lambda x: ', '
    print(top_rules.to_string(index=False))
else:
    print(f"Rules mined: 0")
    print(f"Even with undersampling, no rules generated at these thresholds.
```

```
Rules mined (last fold): 62
Default class: H

Rule count by predicted cause:
predicted_cause
T    27
M    15
E    15
S     4
H     1
Name: count, dtype: int64

Top 15 rules:
                                                                  antecedents pr
edicted_cause  confidence  support     lift
                        Region_South, Track Type_Industry, Visibility_Dark
T    0.791667 0.006859 3.958333
                Region_South, Track Type_Industry, Weather Condition_Cloudy
T    0.750000 0.005415 3.750000
                    Track Type_Main, Visibility_Day, Weather Condition_Rain
M    0.692308 0.006498 3.461538
                             Track Type_Industry, Weather Condition_Cloudy
T    0.644444 0.010469 3.222222
RUCC_Metro_Adjacency_Metro, Track Type_Industry, Weather Condition_Cloudy
T    0.612903 0.006859 3.064516
 RUCC_Metro_Adjacency_Nonmetro_Adjacent, Track Type_Main, Visibility_Dark
E    0.581818 0.011552 2.909091
                                    Track Type_Main, Weather Condition_Rain
M    0.566667 0.012274 2.833333
                                       Track Type_Industry, Visibility_Dark
T    0.561404 0.011552 2.807018
              RUCC_Metro_Adjacency_Metro, Region_South, Track Type_Industry
T    0.551020 0.009747 2.755102
                                  Track Type_Siding, Weather Condition_Clear
T    0.545455 0.006498 2.727273
             Track Type_Industry, Visibility_Dark, Weather Condition_Clear
T    0.540541 0.007220 2.702703
                Track Type_Main, Visibility_Dawn, Weather Condition_Cloudy
E    0.538462 0.005054 2.692308
      RUCC_Metro_Adjacency_Metro, Track Type_Main, Weather Condition_Rain
M    0.534884 0.008303 2.674419
          RUCC_Metro_Adjacency_Metro, Track Type_Industry, Visibility_Dark
T    0.534884 0.008303 2.674419
           RUCC_Metro_Adjacency_Metro, Track Type_Industry, Visibility_Day
T    0.517857 0.010469 2.589286
```

In [31]:

```python
# Per-class metrics
print("Per-class metrics (undersampled CBA, no AT):")
print(results_exp2['per_class_metrics'].to_string(index=False))

print(f"\nClassification report:")
print(classification_report(results_exp2['last_y_val'], results_exp2['last_y
                            labels=CAUSE_ORDER, zero_division=0))

save_experiment(
    results_exp2,
```

```
    features={'categorical': cat_features_exp1, 'numeric': num_features_exp1
    experiment_name='CBA No AT (undersampled)',
    filepath='../results/cba/cba_exp_undersampled.json',
    thresholds={'min_support': 0.005, 'min_confidence': 0.40, 'min_lift': 1.
    undersample=True
)

compare_experiments(
    names=['Baseline (w/ AT)', 'No AT (no undersample)', 'No AT (undersample
    filepaths=[
        '../results/cba/cba_baseline_results.json',
        '../results/cba/cba_exp_no_accident_type.json',
        '../results/cba/cba_exp_undersampled.json'
    ]
)
```

```
Per-class metrics (undersampled CBA, no AT):
Cause  Precision  Recall    F1
    E      0.226   0.221 0.224
    H      0.521   0.671 0.587
    M      0.480   0.417 0.446
    S      0.060   0.129 0.082
    T      0.407   0.207 0.275

Classification report:
              precision    recall  f1-score   support

           E       0.23      0.22      0.22       596
           H       0.52      0.67      0.59      2172
           M       0.48      0.42      0.45      1416
           S       0.06      0.13      0.08       139
           T       0.41      0.21      0.27      1205

    accuracy                           0.44      5528
   macro avg       0.34      0.33      0.32      5528
weighted avg       0.44      0.44      0.43      5528

Saved: ../results/cba/cba_exp_undersampled.json
======================================================================
Experiment                                 CV Mean     CV Std Features
----------------------------------------------------------------------
Baseline (w/ AT)                            0.5001     0.0032        6
No AT (no undersample)                      0.4184     0.0034        5
No AT (undersampled)                        0.4066     0.0167        5
======================================================================
```

Experiment 2 Findings: Undersampling for ARM Rule Mining

The headline: Undersampling produced rules for all 5 cause categories for the first time in any experiment across this entire project.

Cause	Without Undersampling	With Undersampling
Mechanical	41 rules	15 rules

Cause	Without Undersampling	With Undersampling
Track	17 rules	27 rules
Human Factor	2 rules	1 rule
Environmental	**0 rules**	**15 rules**
Signal	**0 rules**	**4 rules**

Similar total rule count (60 vs 62), completely different distribution. Mechanical dropped from dominant to proportional; Environmental and Signal became visible for the first time.

The tradeoff: Overall weighted F1 slipped from 0.418 to 0.407 — majority class accuracy was redistributed toward minority coverage. Environmental jumped from 0.00 to 0.22 F1; Signal from 0.00 to 0.08 F1. That's the expected cost of undersampling: rules mined on balanced data perform differently when deployed against a 16:1 Human Factor to Signal ratio.

The rules have real meaning: "Nonmetro Adjacent + Main track + Dark → Environmental" and "South + Industry track + Dark → Track" describe operational conditions, not shortcuts. No other model in this project can make statements like these.

Higher variance (0.017 vs 0.003) is the known cost of random undersampling — each fold draws a different balanced subset. Worth monitoring but not a reason to abandon the approach.

Experiment 3: Threshold Tuning on Undersampled Data

The thresholds from Experiments 1 and 2 (S=0.005, C=0.40, Lift=1.5) were chosen for imbalanced data with Accident Type. Undersampled data has different statistical properties — each class is now 20% of the training fold, not 2.5–39% — so thresholds that were calibrated for the original distribution may not be optimal here.

What we are doing: Testing multiple support/confidence combinations on the undersampled data to find the best threshold configuration. This is CBA's equivalent of GridSearchCV - instead of searching over model hyperparameters, we search over rule mining thresholds.

In [32]:
```
# Threshold grid search for undersampled CBA
threshold_configs = [
    {'min_support': 0.01,  'min_confidence': 0.30, 'min_lift': 1.5, 'label':
    {'min_support': 0.01,  'min_confidence': 0.40, 'min_lift': 1.5, 'label':
    {'min_support': 0.01,  'min_confidence': 0.50, 'min_lift': 1.5, 'label':
```

```
    {'min_support': 0.005, 'min_confidence': 0.30, 'min_lift': 1.5, 'label':
    {'min_support': 0.005, 'min_confidence': 0.40, 'min_lift': 1.5, 'label':
    {'min_support': 0.005, 'min_confidence': 0.50, 'min_lift': 1.5, 'label':
    {'min_support': 0.05,  'min_confidence': 0.30, 'min_lift': 1.5, 'label':
    {'min_support': 0.05,  'min_confidence': 0.40, 'min_lift': 1.5, 'label':
    {'min_support': 0.05,  'min_confidence': 0.50, 'min_lift': 1.5, 'label':
    {'min_support': 0.10,  'min_confidence': 0.30, 'min_lift': 1.5, 'label':
    {'min_support': 0.10,  'min_confidence': 0.40, 'min_lift': 1.5, 'label':
    {'min_support': 0.10,  'min_confidence': 0.50, 'min_lift': 1.5, 'label':
]

print(f"Testing {len(threshold_configs)} threshold configurations (undersamp
print(f"{'Config':<25} {'CV Mean':>10} {'CV Std':>10} {'Avg Rules':>10}")
print("-" * 58)

threshold_results = []

for config in threshold_configs:
    cba_test = CBAClassifier(
        min_support=config['min_support'],
        min_confidence=config['min_confidence'],
        min_lift=config['min_lift'],
        max_rule_length=4,
        undersample=True
    )

    results = run_cba_experiment(cba_test, X_exp1, y_train)
    avg_rules = np.mean(results['rule_counts'])

    print(f"{config['label']:<25} {results['mean']:>10.4f} {results['std']:>

    threshold_results.append({
        'config': config,
        'results': results,
        'avg_rules': avg_rules
    })

# Find best
best_idx = np.argmax([r['results']['mean'] for r in threshold_results])
best_config = threshold_results[best_idx]
print(f"\nBest: {best_config['config']['label']} "
      f"(F1 = {best_config['results']['mean']:.4f}, "
      f"Avg rules = {best_config['avg_rules']:.1f})")
```

```
Testing 12 threshold configurations (undersampled)...
Config                          CV Mean     CV Std  Avg Rules
------------------------------------------------------------
S=0.01, C=0.30                   0.2629     0.0274      115.0
S=0.01, C=0.40                   0.4119     0.0185       26.6
S=0.01, C=0.50                   0.2877     0.0422        5.2
S=0.005, C=0.30                  0.2682     0.0185      189.6
S=0.005, C=0.40                  0.4171     0.0042       57.6
S=0.005, C=0.50                  0.3334     0.0293       24.0
S=0.05, C=0.30                   0.1963     0.0050       20.0
S=0.05, C=0.40                   0.3182     0.0519        1.4
S=0.05, C=0.50                   0.2218     0.0001        0.0
S=0.10, C=0.30                   0.1963     0.0050        5.4
S=0.10, C=0.40                   0.2218     0.0001        0.0
S=0.10, C=0.50                   0.2218     0.0001        0.0

Best: S=0.005, C=0.40 (F1 = 0.4171, Avg rules = 57.6)
```

In [33]:

```python
# Detailed analysis of best threshold config
best_cfg = best_config['config']
cba_best = CBAClassifier(
    min_support=best_cfg['min_support'],
    min_confidence=best_cfg['min_confidence'],
    min_lift=best_cfg['min_lift'],
    max_rule_length=4,
    undersample=True
)

results_best = best_config['results']

print(f"Best Config: {best_cfg['label']}")
print(f"CV: {results_best['mean']:.4f} +/- {results_best['std']:.4f}")
print(f"\nPer-class metrics:")
print(results_best['per_class_metrics'].to_string(index=False))

print(f"\nClassification report:")
print(classification_report(results_best['last_y_val'], results_best['last_y
                            labels=CAUSE_ORDER, zero_division=0))
```

```
Best Config: S=0.005, C=0.40
CV: 0.4171 +/- 0.0042

Per-class metrics:
Cause  Precision  Recall    F1
    E      0.222   0.230 0.226
    H      0.513   0.623 0.563
    M      0.471   0.400 0.432
    S      0.058   0.201 0.090
    T      0.417   0.205 0.275

Classification report:
              precision    recall  f1-score   support

           E       0.22      0.23      0.23       596
           H       0.51      0.62      0.56      2172
           M       0.47      0.40      0.43      1416
           S       0.06      0.20      0.09       139
           T       0.42      0.20      0.27      1205

    accuracy                           0.42      5528
   macro avg       0.34      0.33      0.32      5528
weighted avg       0.44      0.42      0.42      5528
```

In [34]:

```python
# Refit best config to inspect rules
cba_best.fit(X_exp1, y_train)

if cba_best.n_rules_ > 0:
    print(f"Rules mined (full training set): {cba_best.n_rules_}")
    print(f"Default class: {cba_best.default_class_}")
    print(f"\nRule count by predicted cause:")
    print(cba_best.rules_['predicted_cause'].value_counts())
    print(f"\nTop 15 rules:")
    top_rules = cba_best.rules_[['antecedents', 'predicted_cause', 'confider
                                 'support', 'lift']].head(15).copy()
    top_rules['antecedents'] = top_rules['antecedents'].apply(lambda x: ', '
    print(top_rules.to_string(index=False))
else:
    print("No rules generated with best config.")
```

```
Rules mined (full training set): 51
Default class: H

Rule count by predicted cause:
predicted_cause
T    24
E    13
M    11
S     3
Name: count, dtype: int64

Top 15 rules:
                                                                    antecedents pr
edicted_cause  confidence  support     lift
            Track Type_Industry, Visibility_Day, Weather Condition_Cloudy
T    0.696970 0.006638 3.484848
                     Region_Midwest, Track Type_Industry, Visibility_Day
T    0.677419 0.006061 3.387097
                  Track Type_Main, Visibility_Dark, Weather Condition_Rain
M    0.666667 0.005195 3.333333
          RUCC_Metro_Adjacency_Metro, Region_Midwest, Track Type_Industry
T    0.636364 0.008081 3.181818
                                       Region_Midwest, Track Type_Industry
T    0.626866 0.012121 3.134328
           RUCC_Metro_Adjacency_Nonmetro_Nonadjacent, Track Type_Industry
T    0.625000 0.005772 3.125000
             Region_Midwest, Track Type_Industry, Weather Condition_Clear
T    0.617647 0.006061 3.088235
                             Track Type_Industry, Weather Condition_Cloudy
T    0.600000 0.010390 3.000000
                    Region_South, Track Type_Main, Weather Condition_Rain
M    0.593750 0.005483 2.968750
                     Region_Midwest, Track Type_Industry, Visibility_Dark
T    0.593750 0.005483 2.968750
RUCC_Metro_Adjacency_Metro, Track Type_Industry, Weather Condition_Cloudy
T    0.590909 0.007504 2.954545
                                    Track Type_Main, Weather Condition_Rain
M    0.578947 0.012698 2.894737
      RUCC_Metro_Adjacency_Metro, Track Type_Main, Weather Condition_Rain
M    0.568627 0.008369 2.843137
          RUCC_Metro_Adjacency_Metro, Track Type_Industry, Visibility_Day
T    0.567901 0.013276 2.839506
                   Region_West, Track Type_Main, Weather Condition_Cloudy
E    0.529412 0.005195 2.647059
```

In [35]:

```python
save_experiment(
    results_best,
    features={'categorical': cat_features_exp1, 'numeric': num_features_exp1
    experiment_name='CBA No AT (undersampled & tuned, S=0.005, C=0.40)',
    filepath='../results/cba/cba_tuned_undersampled.json',
    thresholds={
        'min_support': best_cfg['min_support'],
        'min_confidence': best_cfg['min_confidence'],
        'min_lift': best_cfg['min_lift']
    },
    undersample=True,
```

```
    n_configs_tested=len(threshold_configs)
)
```

```
Saved: ../results/cba/cba_tuned_undersampled.json
```

Experiment 3 Findings: Threshold Tuning on Undersampled Data

Best config confirmed: S=0.005, C=0.40, Lift=1.5 — the same thresholds used in Experiment 2. The grid search didn't find anything better; the small F1 improvement (0.4066→0.4171) is within undersampling variance rather than a meaningful threshold gain.

Confidence is the critical parameter. At every support level tested, C=0.40 outperformed both alternatives:

- C=0.30: 115–190 rules, F1 collapses to 0.26–0.27. Too many weak rules fire before reliable ones.
- **C=0.40: 27–58 rules, F1 0.41–0.42.** The sweet spot — rules are reliable enough to be useful but permissive enough to cover minority classes.
- C=0.50: 5–24 rules, F1 0.29–0.33. Too restrictive; most rows fall back to majority class default.

Support has a secondary effect. S=0.005 and S=0.01 performed similarly at C=0.40. At S=0.05 and S=0.10, even balanced data can't produce enough rule-qualifying patterns — at S=0.10/C=0.50, zero rules were generated (F1=0.22, pure majority class predictor).

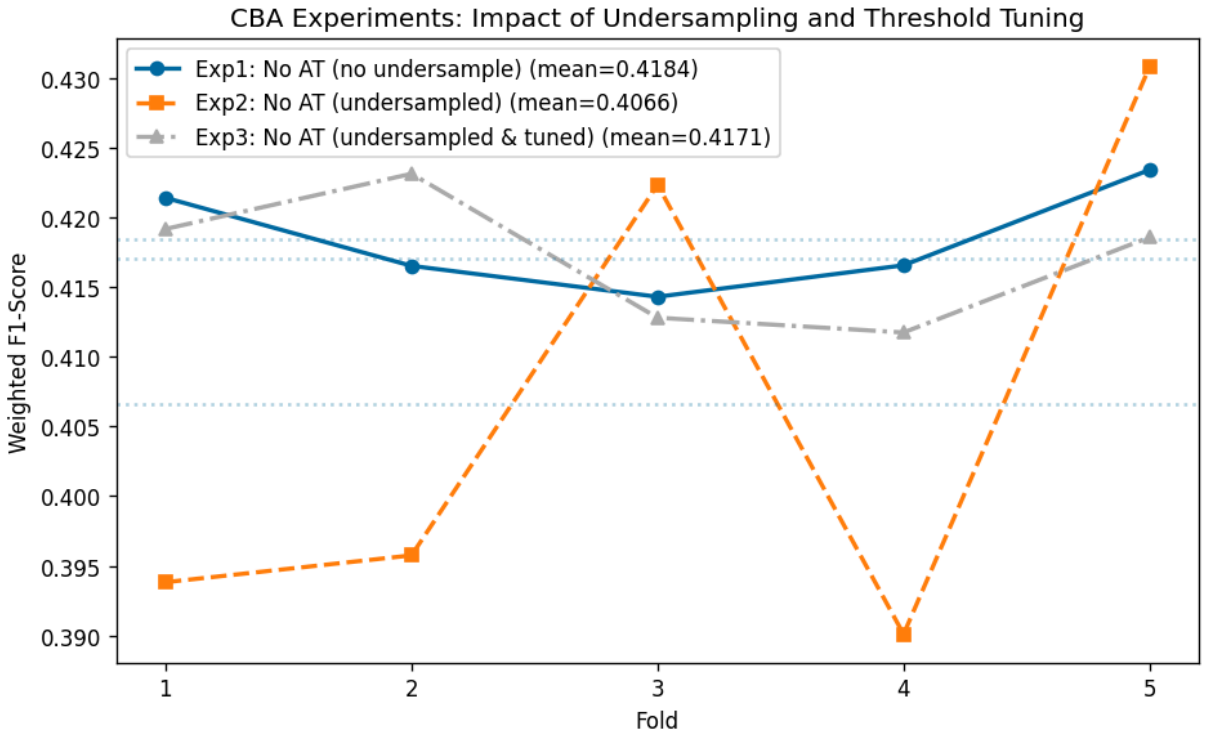
The standout per-class result: Signal recall at 0.20 — the model correctly identifies 1 in 5 Signal accidents. No other model in this project achieves non-trivial Signal recall. Precision is low (0.06), but the patterns are real.

Final rule set (51 rules): Track dominates (24), followed by Environmental (13), Mechanical (11), Signal (3). No Human Factor rules — H predictions come from the majority-class default, which is appropriate. The rules are operationally interpretable: Industry track combinations → Track failures; Rain + Main track → Mechanical; regional + RUCC patterns → Environmental.

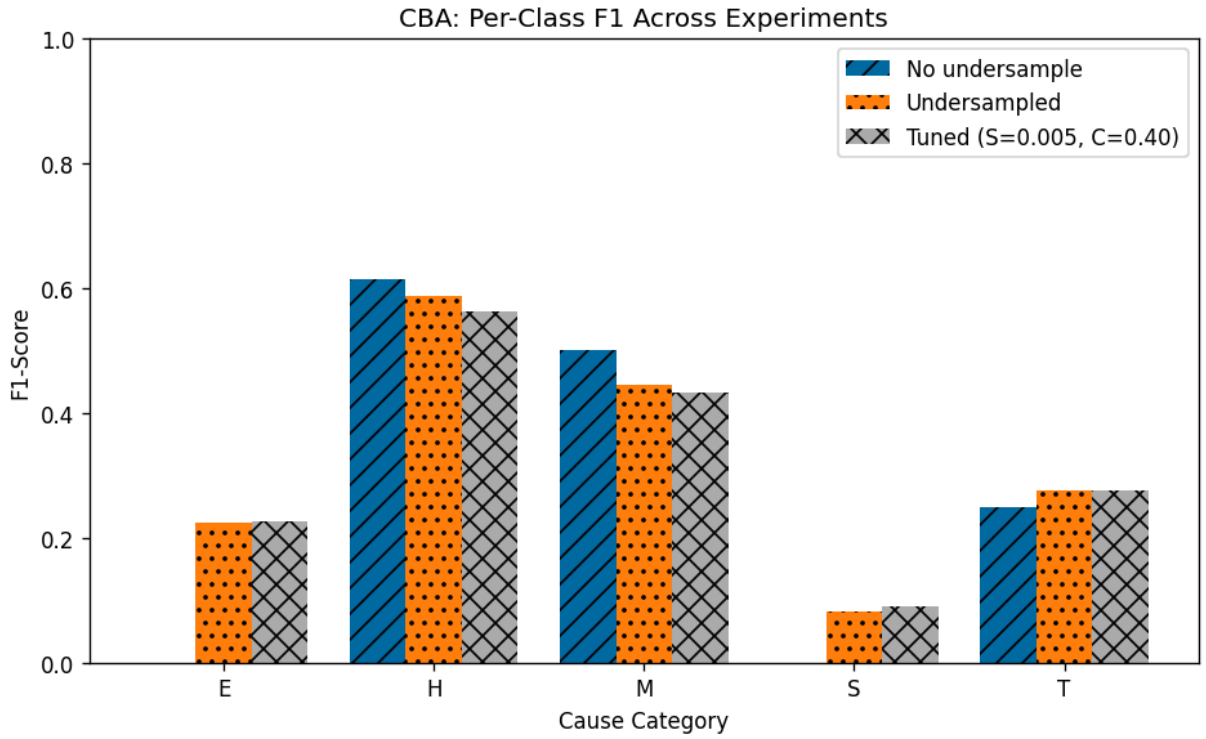
Visualizations

In [36]:
```
# Compare all CBA experiments
plot_cv_comparison(
    experiments=[
        ('Exp1: No AT (no undersample)', results_exp1['scores']),
        ('Exp2: No AT (undersampled)', results_exp2['scores']),
        ('Exp3: No AT (undersampled & tuned)', results_best['scores'])
    ],
```

```
    title='CBA Experiments: Impact of Undersampling and Threshold Tuning'
)
```



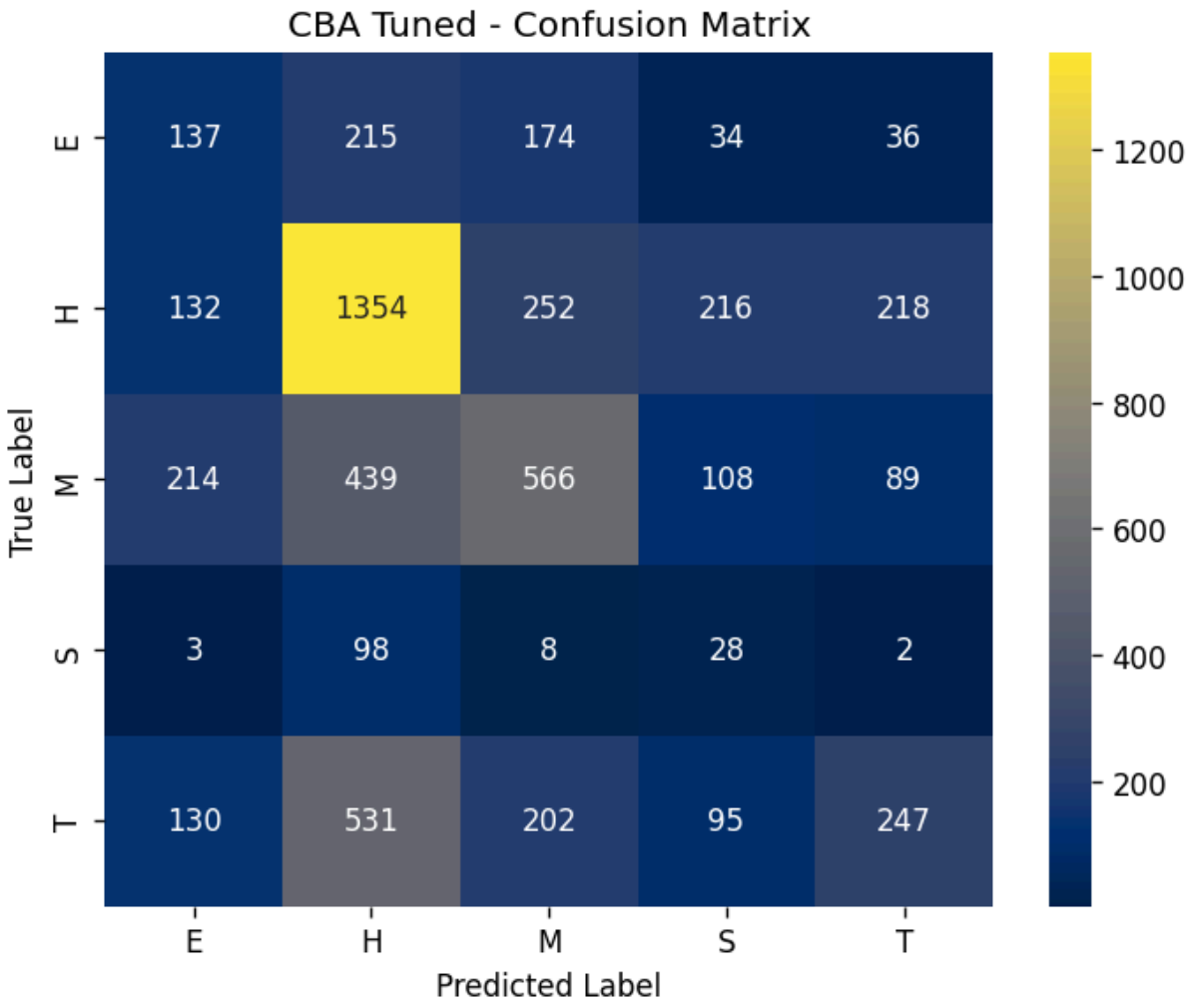
In [37]:
```
# Per-class F1 comparison
plot_per_class_comparison(
    [results_exp1['per_class_metrics'], results_exp2['per_class_metrics'],
     results_best['per_class_metrics']],
    ['No undersample', 'Undersampled', f'Tuned ({best_cfg["label"]})'],
    title='CBA: Per-Class F1 Across Experiments'
)
```



In [38]:
```
# Confusion matrix for best model
cm = confusion_matrix(results_best['last_y_val'], results_best['last_y_pred'
                      labels=CAUSE_ORDER)

fig, ax = plt.subplots(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='cividis',
            xticklabels=CAUSE_ORDER, yticklabels=CAUSE_ORDER, ax=ax)
ax.set_title(f'CBA Tuned - Confusion Matrix')
ax.set_ylabel('True Label')
ax.set_xlabel('Predicted Label')

plt.tight_layout()
plt.show()
```



Save Best Model

In [39]:
```
# Save best CBA model and rules
joblib.dump(cba_best, '../models/cba_tuned_model.pkl')
print(f"Saved: ../models/cba_tuned_model.pkl")

# Save rules to CSV
```

```
if cba_best.n_rules_ > 0:
    rules_export = cba_best.rules_[['antecedents', 'predicted_cause',
                                    'confidence', 'support', 'lift']].copy(
    rules_export['antecedents'] = rules_export['antecedents'].apply(
        lambda x: ', '.join(sorted(x))
    )
    rules_export.to_csv('../data/processed/arm_rules/cba_tuned_rules.csv', i
    print(f"Saved {len(rules_export)} rules to cba_tuned_rules.csv")
```

```
Saved: ../models/cba_tuned_model.pkl
Saved 51 rules to cba_tuned_rules.csv
```

Final Summary

In [40]:

```
# Load and compare all CBA experiments
import os

cba_results_dir = '../results/cba/'
all_files = sorted([f for f in os.listdir(cba_results_dir) if f.endswith('.j

print("=" * 75)
print(f"{'Experiment':<45} {'CV Mean':>10} {'CV Std':>10} {'Features':>8}")
print("-" * 75)

for f in all_files:
    filepath = os.path.join(cba_results_dir, f)
    with open(filepath, 'r') as fh:
        data = json.load(fh)

    name = data.get('experiment', f.replace('.json', ''))
    mean = data.get('cv_mean', 0)
    std = data.get('cv_std', 0)

    if isinstance(data.get('features'), dict):
        n_feat = data['features']['total']
    elif isinstance(data.get('features'), list):
        n_feat = len(data['features'])
    else:
        n_feat = '?'

    print(f"{name:<45} {mean:>10.4f} {std:>10.4f} {n_feat:>8}")

print("=" * 75)
```

```
===========================================================================
Experiment                                    CV Mean     CV Std Features
---------------------------------------------------------------------------
CBA Baseline (with Accident Type)              0.5001     0.0032        6
CBA No AT (no undersampling)                   0.4184     0.0034        5
CBA No AT (undersampled)                       0.4066     0.0167        5
CBA No AT (undersampled & tuned, S=0.005, C=0.40)     0.4171     0.0042
5
===========================================================================
```

Conclusions

Three experiments progressively built toward the best CBA configuration: **S=0.005, C=0.40, Lift=1.5, undersampling enabled, 5 features**. 51 rules covering 4 cause categories, with Human Factor handled by majority-class default.

Key findings:

1. **Removing Accident Type dropped CBA from 0.500 to 0.418 (−0.082)** — a smaller fall than LR (−0.127) or RF (−0.130), but the 60 rules generated still covered only Mechanical, Track, and Human Factor. Signal and Environmental remained invisible at natural class frequencies.

2. **Undersampling solved the minority class problem** — the defining result of the notebook. For the first time across all three models in this project, ARM generated rules for all 5 cause categories. Environmental: 0 rules → 15 rules, 0.00 F1 → 0.22 F1. Signal: 0 rules → 4 rules, 0.00 F1 → 0.09 F1 with 0.20 recall. Overall weighted F1 slipped slightly (0.418 → 0.407) as predictions redistributed from majority to minority classes — the expected tradeoff.

3. **Threshold tuning confirmed S=0.005, C=0.40 was already optimal.** Confidence is the critical parameter: C=0.30 generates too many weak rules (F1 collapses), C=0.50 is too restrictive (too few rules survive), C=0.40 is the balance point.

4. **CBA produces the only interpretable rules in the project.** The fitted model's top rules — "Industry track + Cloudy + Day → Track", "Main track + Dark + Rain → Mechanical", "Nonmetro Adjacent + Main track + Dark → Environmental" — are directly actionable for railroad safety planning. LR and RF cannot produce equivalent explanations.

5. **Cross-model comparison on 5 features without Accident Type:**

Model	CV F1	S Recall	E Recall	Interpretable Rules
LR	0.409	0.00	0.00	No
RF	0.421	0.01	0.04	No
CBA (undersampled)	0.417	0.20	0.23	Yes (51 rules)

CBA is within 0.004 of RF on overall F1 while being the only model that can detect minority causes and explain its predictions.